Tolerance-Aware Control Barrier Functions

The Complete Technical Reference — Theory, Architecture, Training, Inference, and Sim-to-Real Transfer

From a single neuron to a deployed needle-steering controller on the Franka FR3 · FINAL EDITION

Table of Contents

> **Conventions.** Vectors are bold or written componentwise. Positions are in **meters** in the robot base frame. The needle state is $\mathbf{x} \in \mathbb{R}^3 = (x, y, z)$, but $z = 0.44\,\text{m}$ is fixed, so the task is effectively 2-D in (x, y). Code is shown in dark boxes that always wrap to the page width (no horizontal scrolling). Mathematics is typeset; you can read every symbol directly.

Part 0 — The math you need first (zero assumptions)

If you are rusty on linear algebra and calculus, read this part once. Everything later builds on exactly these five ideas: vectors, matrix–vector products, dot products, derivatives/gradients, and differential equations.

0.1 Vectors and norms

A **vector** is an ordered list of numbers, e.g. $\mathbf{x} = (x_1, x_2, x_3)$. Its **Euclidean norm** (length) and squared norm are

$$\|\mathbf{x}\| = \sqrt{x_1^2 + x_2^2 + x_3^2}, \qquad \|\mathbf{x}\|^2 = x_1^2 + x_2^2 + x_3^2.$$

Example. $\mathbf{x} = (0.02, -0.03, 0) \Rightarrow \|\mathbf{x}\|^2 = 0.0004 + 0.0009 + 0 = 0.0013$, $\|\mathbf{x}\| \approx 0.036$.

0.2 Dot product

The **dot product** of two vectors multiplies matching entries and sums them:

$$\mathbf{a}^\top \mathbf{b} = \sum_i a_i b_i.$$

Example. $(-0.85, 0.50, 0) \cdot (0.101, 0.261, 0) = -0.0859 + 0.1305 + 0 = 0.0446$. This single number appears as $\nabla B^\top f_{\text{eff}}$ in the QP.

0.3 Matrix × vector = the linear layer

A **matrix** is a grid of numbers. Multiplying an $m \times n$ matrix W by an n-vector $\mathbf{x}$ produces an m-vector; *row* k of W is dotted with $\mathbf{x}$:

$$(W\mathbf{x})_k = \sum_{j=1}^{n} W_{kj}\, x_j.$$

This is exactly what a neural-network layer does (it then adds a bias and applies an activation). So "a layer resizes a vector from n to m numbers" simply means "multiply by an $m \times n$ matrix."

0.4 Derivative and gradient

The **derivative** $\frac{df}{dz}$ is the slope of f at z: how much f changes per unit change in z. For a function of several variables, the **gradient** collects the partial derivatives into a vector that points in the direction of steepest increase:

$$\nabla f(\mathbf{x}) = \left(\tfrac{\partial f}{\partial x_1}, \tfrac{\partial f}{\partial x_2}, \tfrac{\partial f}{\partial x_3}\right).$$

Example. For $V = \|\mathbf{e}\|^2 = e_1^2 + e_2^2 + e_3^2$, $\frac{\partial V}{\partial e_1} = 2e_1$, so $\nabla V = 2\mathbf{e}$. That is why the code uses $\nabla V = 2\mathbf{e}$ directly.

Two facts we use constantly: (i) moving along $+\nabla f$ increases f fastest; (ii) the chain rule $\frac{d}{dt} f(\mathbf{x}(t)) = \nabla f^\top \dot{\mathbf{x}}$ lets us compute how a certificate changes along a trajectory — this is exactly $\dot{V} = \nabla V^\top \dot{\mathbf{x}}$ and $\dot{B} = \nabla B^\top \dot{\mathbf{x}}$.

0.5 Differential equations (how the needle moves)

A first-order **ordinary differential equation (ODE)** says "velocity = a rule that depends on position": $\dot{\mathbf{x}} = f(\mathbf{x})$. Given a start $\mathbf{x}(0)$ we step forward in small time increments dt. The simplest scheme (Euler) is $\mathbf{x}(t + dt) = \mathbf{x}(t) + dt\, f(\mathbf{x}(t))$; we use the more accurate RK4 (Part IV.2).

0.6 Minimization and constraints (what the QP is)

To "minimize $g(\mathbf{u})$ subject to constraints" means: find the $\mathbf{u}$ that makes g as small as possible while satisfying the listed inequalities. A **Quadratic Program (QP)** is the special case where g is a sum of squares and the constraints are linear — solvable extremely fast, which is why we can run it 100 times per second.

0.7 Symbols used throughout

Symbol	Meaning	Symbol	Meaning
$\mathbf{x}$	needle-tip position (x, y, z)	s	task progress $\in [0, 1]$
f_θ	demo-flow network (velocity)	$\mathbf{e}$	tracking error $\mathbf{x} - \mathbf{x}_{\text{ref}}$
V	Lyapunov (stability) value	B	barrier (safety) value
∇	gradient operator	$\dot{(\cdot)}$	time derivative
$\mathbf{u}$	QP velocity correction	ε	CLF slack ≥ 0
α, γ	CLF / CBF gains	λ	slack penalty
m	runtime barrier margin	β	smooth-min sharpness
Δ	inflation margin (10 mm)	K_f	demo spring gain (=5)

Part I — Foundations & Architecture in Full Detail

I.1 What a single neuron computes

A neuron takes several input numbers, forms a weighted sum, adds a bias, and passes the result through a non-linear *activation* function:

$$y \;=\; \phi\!\left(\sum_{i=1}^{n} w_i\, x_i + b\right) \;=\; \phi(\mathbf{w}^\top \mathbf{x} + b)$$

Here x_i are inputs, w_i are **weights** (learned), b is the **bias** (learned), and ϕ is the activation. The quantity $z = \mathbf{w}^\top \mathbf{x} + b$ is the *pre-activation*.

> **Worked example.** Inputs $\mathbf{x} = (0.5,\ -1.2)$, weights $\mathbf{w} = (2.0,\ -3.0)$, bias $b = 0.5$:
>
> $$z = 2.0(0.5) + (-3.0)(-1.2) + 0.5 = 1.0 + 3.6 + 0.5 = 5.1, \qquad \mathrm{ReLU}(z) = \max(0, 5.1) = 5.1.$$

I.2 The linear (fully-connected) layer

A linear layer applies many neurons at once. With input $\mathbf{x} \in \mathbb{R}^n$, weight matrix $W \in \mathbb{R}^{m \times n}$ and bias $\mathbf{b} \in \mathbb{R}^m$:

$$\mathbf{y} = W\mathbf{x} + \mathbf{b} \in \mathbb{R}^m.$$

"Input dimension" is the length of $\mathbf{x}$; "output dimension" is the length of $\mathbf{y}$. The layer therefore **resizes** a vector from n numbers to m numbers; each output coordinate is one neuron reading all n inputs.

> **Worked example (2 → 3).** $\mathbf{x} = (1, 2)$, $W = \begin{pmatrix} 0.1 & 0.4 \\ -0.2 & 0.3 \\ 0.5 & -0.1 \end{pmatrix}$, $\mathbf{b} = (0,\ 0.1,\ -0.2)$:
>
> $$y_1 = 0.9, \quad y_2 = 0.5, \quad y_3 = 0.1 \;\Rightarrow\; \mathbf{y} = (0.9, 0.5, 0.1).$$

I.3 Activation functions: ReLU and tanh

$$\mathrm{ReLU}(z) = \max(0, z), \qquad \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \in (-1, 1).$$

ReLU zeroes negatives and passes positives unchanged (fast, sparse, no vanishing gradient for $z > 0$). tanh is smooth and bounded in $(-1, 1)$, with derivative $\tanh'(z) = 1 - \tanh^2(z) > 0$ everywhere — essential where we must differentiate B to get ∇B.

Property	ReLU	tanh	Softplus
Formula	$\max(0, z)$	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	$\log(1 + e^z)$

Range	$[0, \infty)$	$(-1, 1)$	$(0, \infty)$
Smooth?	No (kink at 0)	Yes	Yes
Used in	PointNet encoder	f, B MLPs	V correction (forces > 0)

I.4 How information flows; how a layer learns

A node in layer k: (1) receives all activations of layer $k-1$, (2) forms a dot product with its weights, (3) adds bias, (4) applies ϕ, (5) sends its output to all nodes in layer $k+1$. Learning adjusts $W, \mathbf{b}$ by gradient descent to minimize a loss L:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}, \qquad \frac{\partial L}{\partial W^{(k)}} = \frac{\partial L}{\partial \mathbf{a}^{(k)}} \frac{\partial \mathbf{a}^{(k)}}{\partial W^{(k)}} \text{ (chain rule / backprop)}$$

η is the learning rate (e.g. 3×10^{-4}). Backpropagation pushes the gradient from the loss backwards through every layer using the chain rule.

I.5 The state vector $\mathbf{x}$ and normalization

$$\mathbf{x} = (x, y, z), \quad x \in [0.44, 0.55]\,\text{m},\ y \in [0.06, 0.13]\,\text{m},\ z = 0.44\,\text{m (fixed)}.$$

Every network first normalizes its input with the training mean $\boldsymbol{\mu}$ and std $\boldsymbol{\sigma}$:

$$\tilde{\mathbf{x}} = (\mathbf{x} - \boldsymbol{\mu}) \oslash \boldsymbol{\sigma}$$

```
# Example normalization
mu  = [0.490, 0.080, 0.440]
sig = [0.028, 0.018, 0.003]
x   = [0.500, 0.100, 0.440]
x_tilde = [(0.500-0.490)/0.028, (0.100-0.080)/0.018, 0.0]
        = [0.357, 1.111, 0.000]
```

I.6 Task progress $s \in [0, 1]$

The demo path is stored as $R = 200$ waypoints. Progress is the normalized index of the nearest waypoint, and the reference position is a linear interpolation:

$$s = \frac{\arg\min_k \|\mathbf{x} - \mathbf{p}_k\|}{R - 1}, \qquad \mathbf{x}_{\text{ref}}(s) = (1 - w)\,\mathbf{p}_{\lfloor i \rfloor} + w\,\mathbf{p}_{\lceil i \rceil}, \quad i = s(R-1).$$

I.7 Component A — Demo-flow network $f_\theta(\mathbf{x}, s)$

This outputs the desired velocity. It is a **plain 4-layer MLP** (not a ResNet) with tanh, followed by a spring term toward the demo path:

$$f_\theta(\mathbf{x}, s) = \underbrace{v_{\text{net}}(\tilde{\mathbf{x}}, s)}_{\text{MLP}} + \underbrace{K_f\big(\mathbf{x}_{\text{ref}}(s) - \mathbf{x}\big)}_{\text{feedback spring}}, \qquad K_f = \texttt{DEMO\_K} = 5.$$

```
# f_theta architecture (config: F_HIDDEN = [128,128,128])
input  : [x_tilde(3), s(1)]            -> R^4
Linear(4   -> 128) + Tanh              -> h1 in R^128
Linear(128 -> 128) + Tanh              -> h2 in R^128
Linear(128 -> 128) + Tanh              -> h3 in R^128
Linear(128 -> 3)   (no activation)     -> v_net in R^3   (velocity, any sign)
params = 640 + 16,512 + 16,512 + 387 = 34,051
```

Layer	In	Out	What it represents
Linear+tanh	4	128	non-linear features of position & progress
Linear+tanh	128	128	"am I in the upward-curving segment near $s = 0.3$?"
Linear+tanh	128	128	turning direction / curvature features
Linear	128	3	final 3-D velocity (unbounded)

Worked example. $\mathbf{x} = (0.500, 0.100, 0.44)$, $s = 0.5$. Suppose the MLP returns $v_{\text{net}} = (0.015, -0.005, 0)$ and $\mathbf{x}_{\text{ref}}(0.5) = (0.494, 0.091, 0.44)$. Then

$$K_f(\mathbf{x}_{\text{ref}} - \mathbf{x}) = 5(-0.006, -0.009, 0) = (-0.030, -0.045, 0),$$
$$f_\theta = (0.015 - 0.030,\ -0.005 - 0.045,\ 0) = (-0.015, -0.050, 0)\ \text{m/s}.$$

I.8 Component B — the FULL Lyapunov network V_θ

The implemented V is **not** simply $\|e\|^2$. It is a quadratic base times a small, strictly-positive learned correction:

$$V(\mathbf{x}, s) = \|\mathbf{e}\|^2 \big(1 + \delta \cdot \text{corr}(\tilde{\mathbf{e}}, s)\big), \qquad \mathbf{e} = \mathbf{x} - \mathbf{x}_{\text{ref}}(s),\ \ \delta = 0.05.$$

```
# corr() is a Softplus MLP (config: V_HIDDEN = [64,64])
input : [e_tilde(3), s(1)]             -> R^4
Linear(4  -> 64) + Softplus            -> R^64
Linear(64 -> 64) + Softplus            -> R^64
Linear(64 -> 1)  + Softplus            -> R^1   (>= 0 ALWAYS)
params = 320 + 4,160 + 65 = 4,545
```

Because every layer uses Softplus, corr ≥ 0, hence $1 + \delta\,\text{corr} \geq 1$, so $V \geq \|e\|^2 > 0$ for $\mathbf{e} \neq 0$. The correction reshapes the circular bowl into a mild ellipsoid that matches the corridor, but never dominates ($\delta = 0.05$).

Key property ($V = 0$ exactly on the path). When $\mathbf{x} = \mathbf{x}_{\text{ref}}(s)$, $\mathbf{e} = \mathbf{0} \Rightarrow \|e\|^2 = 0 \Rightarrow V = 0$ regardless of corr, because the correction is *multiplicative*.

I.9 Component C — the Composite Barrier B_φ

$B_\varphi(\mathbf{x})$ measures safety against *all* obstacles at once. It is built from three parts: a PointNet encoder (A), a shared conditional CBF MLP (B), and a smooth-min fusion (C).

Part A — ObstacleEncoder (PointNet)

Input is a point cloud $P_i \in \mathbb{R}^{K\times 2}$ ($K = 200$ interior points, centered at the obstacle centroid). The same per-point MLP is applied to every point, then a max-pool aggregates:

$$\mathbf{h}_j = \mathrm{MLP}(\mathbf{p}_j) \in \mathbb{R}^{64} \;\; (j = 1..K), \qquad \mathbf{e} = \max_{j=1..K} \mathbf{h}_j \;\; (\text{elementwise}) \in \mathbb{R}^{64}.$$

```
# Per-point MLP (shared weights), then max-pool
for each point p_j in R^2:
    Linear(2   -> 128) + ReLU
    Linear(128 -> 128) + ReLU
    Linear(128 -> 64)   (no activation)   -> h_j in R^64
e[k] = max_j h_j[k]   for k = 0..63          # permutation-invariant
encoder params ~ 25,152
```

Single-point trace (star point $\mathbf{p}_1 = (0.003, -0.008)$, one hidden unit):

$$z = \mathbf{w}_7^\top \mathbf{p}_1 + b_7 = 0.42(0.003) + (-1.15)(-0.008) + 0.03 = 0.040, \quad \mathrm{ReLU}(0.040) = 0.040.$$

A different unit with $z = -0.023$ is killed: $\mathrm{ReLU}(-0.023) = 0$. About half of the 128 units fire; the rest are zero. After three layers we obtain $\mathbf{h}_1 \in \mathbb{R}^{64}$.

Part B — ConditionalObstacleCBF (shared MLP)

One MLP serves every obstacle. Obstacle identity enters only through the embedding $\mathbf{e}_i$ and the relative coordinate:

$$\mathbf{x}_{\mathrm{rel}} = \mathbf{x} - \mathbf{c}_i, \qquad b_i(\mathbf{x}) = \mathrm{MLP}_{\mathrm{cbf}}\big([\,\mathbf{x}_{\mathrm{rel}} \oslash \boldsymbol{\sigma},\ \mathbf{e}_i\,]\big) \in \mathbb{R}.$$

```
# Input dim = 3 (x_rel/sigma) + 64 (embedding) = 67
Linear(67  -> 256) + Tanh
Linear(256 -> 256) + Tanh
Linear(256 -> 256) + Tanh
Linear(256 -> 1)    (no activation)   -> b_i in R^1
last layer init ~ U(-0.01, +0.01), bias 0   # b_i(x) ~ 0 at start of training
cbf-MLP params ~ 149,249
```

Sign convention: $b_i > 0$ outside obstacle i, $b_i = 0$ on the inflated boundary ($\approx$ 10mm outside the true tissue), $b_i < 0$ inside the keep-out zone.

Part C — Smooth-min fusion

Safety requires being safe from every obstacle, i.e. $B = \min_i b_i$. The non-smooth min is replaced by the differentiable log-sum-exp (CN-CBF Eq. 18):

$$B(\mathbf{x}) = -\frac{1}{\beta}\log\sum_{i=1}^{N} e^{-\beta\, b_i(\mathbf{x})}, \qquad \beta = 1000.$$

Worked example with $b = (0.015, 0.003, 0.040)$, $\beta = 1000$:

$$\textstyle\sum_i e^{-\beta b_i} = e^{-15} + e^{-3} + e^{-40} \approx e^{-3} = 0.0498,$$

$$B = -\tfrac{1}{1000}\log(0.0498) \approx 0.003 = \min(0.015, 0.003, 0.040). \checkmark$$

The gradient ∇B is obtained by autograd through encoder → CBF → smooth-min, and it points away from the nearest obstacle.

Part II — PointNet: shape encoding & permutation invariance

II.1 What a point cloud is

A point cloud is an *unordered* set of points sampling a shape. The star's interior is sampled into $K = 200$ coordinates centered at its centroid:

```
P_star = [[ 0.003, -0.008],   # point 1
          [-0.005,  0.012],   # point 2
          [ 0.010,  0.001],   # ...
          ...  200 points ... ]   # shape (200, 2)
```

II.2 How the cloud is sampled (rejection sampling)

```
def canonical_interior_cloud(shape, k=96):
    pts = []
    while len(pts) < k:
        p = center + uniform(-1.8*R, +1.8*R, size=2)   # R = bounding radius
        if sdf_critical_shape_2d(p, shape) < 0:           # inside?
            pts.append(p - center)                       # store centered
    return array(pts)   # sampled ONCE; transform R*scale for any pose later
```

II.3 Why max-pool gives permutation invariance

For any permutation π of the points,

$$\max\{\mathbf{h}_{\pi(1)}, \ldots, \mathbf{h}_{\pi(K)}\} = \max\{\mathbf{h}_1, \ldots, \mathbf{h}_K\}.$$

The maximum ignores order, so the embedding $\mathbf{e}$ is identical no matter how the cloud is listed — exactly what we need for sensor data that returns points in arbitrary order. (There is **no** k-means clustering anywhere; max-pool replaces it.)

II.4 A fully numeric forward pass (tiny example)

To make the data flow concrete, here is a complete pass with a tiny encoder ($2 \to 3 \to 2$, max-pool over 3 points). The real network is $2 \to 128 \to 128 \to 64$ over 200 points; only the sizes differ.

```
# Points (3 of them), centered at obstacle centroid
p1 = (0.30, 0.20)   p2 = (0.10, 0.40)   p3 = (0.50, 0.05)

# Layer 1: Linear(2 -> 3) + ReLU,  W1 rows = features
W1 = [[ 1.0,  0.0],     b1 = [0.00,
      [ 0.0,  1.0],           0.00,
      [ 1.0,  1.0]]           -0.20]

# pre-activations z = W1.p + b1, then ReLU
p1: z = (0.30, 0.20, 0.30+0.20-0.20=0.30) -> ReLU -> h1 = (0.30, 0.20, 0.30)
p2: z = (0.10, 0.40, 0.10+0.40-0.20=0.30) -> ReLU -> h2 = (0.10, 0.40, 0.30)
p3: z = (0.50, 0.05, 0.50+0.05-0.20=0.35) -> ReLU -> h3 = (0.50, 0.05, 0.35)
```

```
# Layer 2: Linear(3 -> 2), no activation (last per-point layer)
W2 = [[1.0, 0.0, 0.0],   b2 = [0.0,
      [0.0, 0.0, 1.0]]          0.0]
g1 = (0.30, 0.30)   g2 = (0.10, 0.30)   g3 = (0.50, 0.35)

# Max-pool over the 3 points, elementwise
e[0] = max(0.30, 0.10, 0.50) = 0.50     # point p3 wins dim 0
e[1] = max(0.30, 0.30, 0.35) = 0.35     # point p3 wins dim 1
embedding e = (0.50, 0.35)
```

Notice the embedding depends only on the *set* of points, not their order — shuffle p1,p2,p3 and the max is identical. This little $\mathbf{e}$ would then be concatenated with the relative position and fed to the conditional-CBF MLP.

II.5 How a layer actually learns (concrete backprop)

Suppose the network output is $\hat{y}$ and the target is y, with squared-error loss $L = (\hat{y} - y)^2$. For a single weight w feeding the output through $\hat{y} = \phi(wx + b)$:

$$\frac{\partial L}{\partial w} = \underbrace{2(\hat{y} - y)}_{\partial L/\partial \hat{y}} \cdot \underbrace{\phi'(wx + b)}_{\text{local slope}} \cdot \underbrace{x}_{\partial z/\partial w}.$$

Example. $x = 2$, $w = 0.5$, $b = 0$, $\phi = \mathrm{ReLU}$, target $y = 1$. Then $z = 1$, $\hat{y} = \mathrm{ReLU}(1) = 1$, so $L = (1 - 1)^2 = 0$ and $\partial L/\partial w = 0$ — already correct. If instead $w = 0.1$: $z = 0.2, \hat{y} = 0.2, L = (0.2 - 1)^2 = 0.64$, $\partial L/\partial w = 2(0.2 - 1) \cdot 1 \cdot 2 = -3.2$; with $\eta = 0.01$, $w \leftarrow 0.1 - 0.01(-3.2) = 0.132$ — nudged upward to reduce the loss.

Backprop chains this rule layer by layer (the "$\partial L/\partial \hat{y}$" of one layer becomes the incoming gradient of the layer below). That is the entire learning mechanism.

II.6 What the 64 embedding dimensions mean

Suppose dimension $k = 15$ encodes "pointiness toward the upper-right." The per-point MLP scores each point's contribution; the max-pool keeps the most extreme one. So $\mathbf{e}[15]$ answers "what is the *most* upper-right-pointy feature anywhere in this obstacle?" A star (5 sharp tips) scores high; a blob (smooth) scores low. This is how one shared encoder distinguishes star, crescent, blob, kidney, and L-shape.

Part III — Lyapunov theory & the stability guarantee

III.1 The CLF decrease condition

$$\dot{V} = \nabla V^{\top} \dot{\mathbf{x}} \le -\alpha V, \qquad \alpha = \texttt{ALPHA\_CLF} = 4.$$

Treating this as an ODE in $V(t)$ and applying Grönwall's inequality:

$$\frac{d}{dt} \log V \le -\alpha \;\Rightarrow\; V(t) \le V(0)\, e^{-\alpha t} \;\Rightarrow\; \|\mathbf{e}(t)\| \le \|\mathbf{e}(0)\|\, e^{-\alpha t/2}.$$

With $\alpha = 4$, a 20 mm initial error decays below 0.7 mm in about 1.7 s. In code the gradient is the closed form $\nabla(\|e\|^2) = 2\mathbf{e}$ (the correction's gradient $\propto \delta$ vanishes as $\mathbf{e} \to 0$, where the QP operates).

III.2 Where the guarantee holds — honestly

Local vs global. The QP enforces $\dot{V} \le -\alpha V$ *whenever it is feasible*.

- **Near the path** (small V): CLF and CBF rarely conflict → exponential convergence holds.
- **Near an obstacle:** the hard CBF may force motion *away* from the path, momentarily raising V; the soft CLF is relaxed by slack ε so the QP stays feasible.
- **Path blocked (pocket trap):** a purely reactive controller has no global plan; the only joint CLF+CBF solution may be to stop. This is the fundamental limit addressed by scene validation, not by the certificates themselves.

Part IV — The complete training pipeline

IV.1 Two stages

```
STAGE 1 (train.py, ~2.5 h GPU):
  Phase 0  : pretrain f_theta alone, 300 epochs, RK4 rollout MSE
  Outer loop (OUTER_ITERS=10):
     Inner loop (INNER_EPOCHS=150): jointly train f, V, B (barrier weight ramps 0->1 over iters 0..2)
     sample N_CEX=2000 counter-examples; add violations to the training sets
     if iter>=3 and no violations -> converged

STAGE 2 (train_barrier_fast.py, ~3-5 min GPU):
  load f,V from Stage 1; FREEZE them; re-init B
  train ONLY B for 12,000 steps with pose/scale/rotation augmentation
```

IV.2 Phase 0 — imitation via Neural ODE (RK4)

$$L_{\text{MSE}} = \frac{1}{MT}\sum_{i=1}^{M}\sum_{k=1}^{T}\left\|\hat{\mathbf{x}}_i(t_k) - \mathbf{x}_i(t_k)\right\|^2,$$

where $\hat{\mathbf{x}}$ is obtained by integrating $\dot{\mathbf{x}} = f_\theta(\mathbf{x}, s)$ with the 4th-order Runge–Kutta scheme:

$$\begin{aligned}
\mathbf{k}_1 &= f_\theta(\mathbf{x}, s), & \mathbf{k}_2 &= f_\theta(\mathbf{x} + \tfrac{dt}{2}\mathbf{k}_1,\ s),\\
\mathbf{k}_3 &= f_\theta(\mathbf{x} + \tfrac{dt}{2}\mathbf{k}_2,\ s), & \mathbf{k}_4 &= f_\theta(\mathbf{x} + dt\,\mathbf{k}_3,\ s),
\end{aligned}
\qquad \mathbf{x}_{k+1} = \mathbf{x}_k + \tfrac{dt}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4).$$

RK4 evaluates f_θ four times per step; its local error is $O(dt^5)$, far better than Euler's $O(dt^2)$.

A numeric RK4 step

```
# Suppose at x=(0.46,0.085,0.44), s=0.3, dt=0.02, and f returns these slopes:
k1 = f(x,            s)      = ( 0.40,  0.15, 0)   # m/s
k2 = f(x+dt/2*k1,    s)      = ( 0.42,  0.10, 0)
k3 = f(x+dt/2*k2,    s)      = ( 0.41,  0.11, 0)
k4 = f(x+dt*k3,      s)      = ( 0.43,  0.06, 0)
x_next = x + dt/6*(k1 + 2k2 + 2k3 + k4)
       = x + 0.02/6*( (0.40+0.84+0.82+0.43), (0.15+0.20+0.22+0.06), 0 )
       = x + 0.003333*( 2.49, 0.63, 0 )
       = (0.46+0.00830, 0.085+0.00210, 0.44)
       = (0.46830, 0.08710, 0.44)
```

The weighted average $(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)/6$ is far more accurate than using $\mathbf{k}_1$ alone (Euler), which is why training rollouts use RK4.

IV.3 The SDF-shaped barrier target

$$b_{\text{target}}(\mathbf{x}) = K \cdot \text{clip}\big(\text{sdf}(\mathbf{x}) - \Delta,\ -\text{CLAMP}_{\text{in}},\ +\text{CLAMP}_{\text{out}}\big),$$

with $K = 4$, $\Delta = \texttt{INFLATE\_MARGIN} = 0.010$ m, $\text{CLAMP}_{\text{in}} = 0.040$ m, $\text{CLAMP}_{\text{out}} = 0.013$ m. The asymmetric clamp keeps ∇B alive deep inside the obstacle (wide interior band) while bounding the exterior ceiling.

```
# numeric targets
sdf = 0.025 m -> 4*clip(0.015, -0.040, +0.013) = 4*0.013 = +0.052   (clamped ceiling)
sdf = 0.012 m -> 4*clip(0.002, ...) = +0.008
sdf = 0.005 m -> 4*clip(-0.005,...) = -0.020   (inside inflation -> negative)
```

Because $b = 0$ is trained at sdf $= \Delta = 10$mm, "$B \geq 0$" already means "≥10 mm outside the true tissue."

IV.4 The three core loss terms

$$\begin{aligned} L_{\text{safe}} &= \tfrac{1}{N_s} \sum \big(B_\varphi(\mathbf{x}) - b_{\text{target}}(\mathbf{x})\big)^2 && (\lambda_{\text{FS}} = 5), \\ L_{\text{unsafe}} &= \tfrac{1}{N_u} \sum \max\big(0,\ B_\varphi(\mathbf{x}) + 0.001\big)^2 && (\lambda_{\text{OB}} = 12), \\ L &= \tfrac{1}{N} \sum \max\big(0,\ \nabla V^\top f_\theta + \alpha V\big)^2 && (\lambda_{\text{DOT}} = 0.5). \end{aligned}$$

L_{safe} regresses B onto the SDF shape; L_{unsafe} is a one-sided hinge that forces $B < 0$ inside (weighted heavily because misses are dangerous); L trains the demo flow to be Lyapunov-compatible.

IV.5 Counter-example (CEX) refinement

```
# after each inner loop, sample 2000 random points and flag failures
add x to CEX buffer  if  B(x) > B_SAFE_MARGIN  but  sdf(x) < INFLATE
# next inner loop: 20% of each minibatch is drawn from CEX -> fix worst mistakes
```

IV.6 Stage 2 — pose/scale augmentation (zero-shot generalization)

$$\theta \sim \mathcal{U}(-\pi, \pi), \quad \sigma \sim \mathcal{U}(0.65, 1.4), \quad \mathbf{t} \sim \mathcal{U}(-0.05, 0.05)^2,$$

$$R = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}, \quad \mathbf{p}_{\text{aug}} = \sigma R\,\mathbf{p} + \mathbf{t}, \quad \text{sdf}_{\text{aug}}(\mathbf{x}) = \sigma\,\text{sdf}\big(R^{-1}(\mathbf{x} - \mathbf{t})/\sigma\big).$$

The *same* transform is applied to the cloud and to the SDF labels, so the network must learn pose-invariant features. After 12,000 random poses it generalizes to all poses.

> **Why freeze f and V?** They do not depend on obstacle pose — rotating an obstacle does not move the demo path. Letting them train under augmentation would only inject noise that corrupts the demo-following and Lyapunov behavior learned in Stage 1. Freezing also makes Stage 2 ~3× faster (only B's 149k params update).

IV.7 The eikonal (gradient-cap) term — what it fixes

The "near-step" failure. The hinge alone forced $B < 0$ inside but not a sane slope. The network cheated by learning a near step function ($\|\nabla B\| \approx 290\,\mathrm{m}^{-1}$). Then in the QP, tiny angle changes flipped `cbf_rhs` between large positive and negative values, occasionally yielding $u = 0$ so the demo spring drove the needle straight through.

$$L_{\text{eik}} = \lambda_{\text{eik}} \frac{1}{N} \sum \mathrm{ReLU}\big(\|\nabla B_\varphi(\mathbf{x})\| - \texttt{B\_GRAD\_MAX}\big)^2, \quad \texttt{B\_GRAD\_MAX} = 12,\ \lambda_{\text{eik}} = 0.02.$$

With the SDF target giving $\|\nabla B\| \approx K = 4$ and the cap at $3K = 12$, the barrier becomes a smooth scaled SDF and the QP is well-conditioned. **This term is unrelated to** m — m is the runtime barrier margin in the CBF constraint (Part V); the eikonal term is a training regularizer on the gradient magnitude.

$$L_{\text{Stage 2}} = \lambda_{\text{reg}}(L_{\text{reg}} + L_{\text{comp}}) + \lambda_{\text{FS}} L_{\text{safe}} + \lambda_{\text{OB}} L_{\text{unsafe}} + \lambda_{\text{DOT}} L + \lambda_{\text{eik}} L_{\text{eik}}.$$

IV.8 One stage vs two stages — the deeper "why"

A natural question: why not train f, V, B together with augmentation and counter-examples in a single stage? The table contrasts the two designs.

Aspect	Single joint stage (with aug.)	Two stages (what we do)
Gradient interference	barrier-augmentation gradients perturb f, V; demo-following degrades	f, V protected once frozen; clean barrier optimization
What f, V depend on	—	only the demo path (pose-independent) → no reason to re-train under aug.
Speed	every step pays RK4 imitation + barrier; hours per change	Stage 2 skips RK4, updates 149k params only → ~5 min
Iterating on B	any barrier tweak re-runs the whole pipeline	re-run Stage 2 alone; f, V reused
Stability of training	optimizer "fights itself" (B vs f gradients)	each objective optimized in isolation
When single-stage is fine	tiny problems where everything co-converges	—

Stage 1 still includes the barrier (ramped in) so that f becomes Lyapunov- and barrier-aware; Stage 2 then specializes B for pose generalization without risking the now-good f, V. It is a deliberate "coarse joint, then fine specialize" schedule.

IV.9 Training diagnostics & debugging

Symptom	Likely cause	Fix
Needle passes through obstacles	near-step B ($\|\nabla B\|$ huge)	ensure SDF target + eikonal cap active; check ‖∇B‖≈4–12

$B > 0$ inside obstacles	unsafe hinge weight λ_{OB} too low	raise λ_{OB} (=12); add CEX replay
Best checkpoint has untrained B	best chosen before barrier weight ramped in	prefer final_model.pt; guard best-tracking by bar_weight>0
Standoff too large (over-conservative)	barrier "compression" (slope<K)	accept & size scene clearance, or improve edge accuracy
QP infeasible / erratic u	conflicting CBF/CLF, tiny ‖∇B‖	slack on CLF (already soft); project_safe guards small ‖∇B‖
Needle stalls head-on	∇B opposes f_θ	enable swirl (SWIRL_GAIN=1)
Generalization fails on rotation	encoder embedding OOD	confirm Stage-2 pose/scale augmentation ran

Part V — Online inference: the QP at 100 Hz

V.1 The optimization problem

$$\min_{\mathbf{u},\,\varepsilon\geq 0} \;\|\mathbf{u}\|^2 + \lambda\,\varepsilon^2$$

$$\text{s.t.}\quad \nabla B^\top (f_{\text{eff}} + \mathbf{u}) \geq -\gamma\,(B - m) \quad [\text{CBF, hard}],$$

$$\nabla V^\top (f_{\text{eff}} + \mathbf{u}) \leq -\alpha V + \varepsilon \quad [\text{CLF, soft}].$$

with $\gamma = 3$, $\alpha = 4$, $\lambda = 0.5$, and $m = \texttt{B\_SAFE\_MARGIN} = 0.008$. Here m **is the safety margin**: the controller defends the level set $\{B \geq m\}$ (not $\{B \geq 0\}$) to absorb residual barrier error. $f_{\text{eff}} = f_\theta + \mathbf{g}$ includes the optional go-around guidance $\mathbf{g}$.

V.2 OSQP standard form

$$\min_{\mathbf{z}} \;\tfrac{1}{2}\mathbf{z}^\top P\mathbf{z} + q^\top\mathbf{z} \quad \text{s.t. } l \leq A\mathbf{z} \leq u, \qquad \mathbf{z} = [\mathbf{u};\varepsilon].$$

```
P = diag(2, 2, 2, 2*lambda)      # 0.5*z'Pz = |u|^2 + lambda*eps^2
q = [0, 0, 0, 0]
A = [[gradB_x, gradB_y, gradB_z,  0 ],   # CBF row
     [gradV_x, gradV_y, gradV_z, -1 ],   # CLF row
     [   0,       0,       0,      1 ]]   # eps >= 0
l = [cbf_rhs, -inf, 0]
u = [+inf, clf_rhs, +inf]
cbf_rhs = -gamma*(B - m) - gradB . f_eff
clf_rhs = -alpha*V       - gradV . f_eff
```

V.3 How OSQP solves it (ADMM, in plain terms)

OSQP uses **ADMM** (Alternating Direction Method of Multipliers). It splits the problem into two easy pieces and bounces between them until they agree:

1. **Unconstrained quadratic step** — solve a linear system (a factorization reused every step) to minimize the cost ignoring the inequality bounds.
2. **Projection step** — clip the result back into the feasible box $l \leq A\mathbf{z} \leq u$.
3. **Dual update** — adjust multipliers that "pull" the two steps toward consistency.

Each pass is cheap; with `warm_starting=True` the previous tick's solution is the starting guess, so only ~10–50 passes are needed and the whole solve takes well under a millisecond. "Polishing not needed" in the logs just means the solution was already accurate — it is not an error.

$$\text{status} \in \{\text{solved}, \text{solved\_inaccurate}\} \Rightarrow \mathbf{z}^\star = [\mathbf{u}^\star; \varepsilon^\star]; \qquad \text{else fallback: } \mathbf{u} = \frac{\max(0, \text{cbf\_rhs})}{\|\nabla B\|^2} \nabla B.$$

The fallback (used only if the QP fails) is the minimum-norm $\mathbf{u}$ that meets the CBF constraint alone — safety first, convergence second.

V.4 A complete worked control step

```
# Scenario: needle near the star, head-on
x = (0.498, 0.062, 0.440)        star center = (0.500, 0.050, 0.440)
s = 0.556                          (nearest demo waypoint index 5/9)
```

Step 2 — demo flow.

$$v_{\text{net}} = (0.025, 0.030, 0), \quad \mathbf{x}_{\text{ref}} = (0.503, 0.091, 0.44), \quad f_\theta = v_{\text{net}} + 5(\mathbf{x}_{\text{ref}} - \mathbf{x}) = (0.050,\ 0.175,\ 0)\ \text{m/s}.$$

Step 3 — barrier. Star dominates the smooth-min: $B = 0.018$, $\nabla B = (-0.85, 0.50, 0)$.

Step 4 — CLF.

$$\mathbf{e} = (-0.005, -0.029, 0), \quad V = \|e\|^2 = 8.66 \times 10^{-4}, \quad \nabla V = 2\mathbf{e} = (-0.010, -0.058, 0).$$

Step 5 — go-around guidance (active since $B < \texttt{B\_ACTIVE} = 0.040$).

$$\mathbf{n} = \frac{\nabla B}{\|\nabla B\|} = (-0.862, 0.507, 0), \quad \mathbf{t} = (-n_y, n_x, 0) \xrightarrow{\text{flip toward goal}} (0.507, 0.862, 0),$$

$$\text{closeness} = \frac{0.040 - 0.018}{0.040} = 0.55, \quad \mathbf{g} = 1 \cdot \|f_\theta\| \cdot 0.55 \cdot \mathbf{t} = (0.051, 0.086, 0),$$

$$f_{\text{eff}} = f_\theta + \mathbf{g} = (0.101,\ 0.261,\ 0), \qquad \nabla B^\top \mathbf{g} \approx 0 \ \text{(tangential)}.$$

Step 6 — right-hand sides.

$$\text{cbf\_rhs} = -3(0.018 - 0.008) - \nabla B^\top f_{\text{eff}} = -0.030 + 0.0446 = -0.075,$$

$$\text{clf\_rhs} = -4(8.66 \times 10^{-4}) - \nabla V^\top f_{\text{eff}} = +0.0127.$$

Step 7 — instantiate the exact QP matrices.

```
# z = [u_x, u_y, u_z, eps];  gradB=(-0.85,0.50,0), gradV=(-0.010,-0.058,0)
P = diag(2, 2, 2, 1.0)            # 2*lambda = 2*0.5 = 1.0
q = [0, 0, 0, 0]
A = [[-0.85,  0.50, 0.0,  0.0],   # CBF
     [-0.010,-0.058,0.0, -1.0],   # CLF
     [ 0.0,   0.0,  0.0,  1.0]]   # eps>=0
```

```
l = [-0.075, -inf, 0.0]
u = [ +inf, +0.0127, +inf]
# CBF row: -0.85 u_x + 0.50 u_y >= -0.075   (already true at u=0: 0 >= -0.075)
# CLF row: -0.010 u_x -0.058 u_y - eps <= 0.0127 (true at u=0,eps=0: 0 <= 0.0127)
```

Step 8 — solve. cbf_rhs $= -0.075 < 0$ means f_{eff} already satisfies the CBF, so OSQP returns $\mathbf{u} \approx \mathbf{0}$, $\varepsilon \approx 0$. The swirl alone steers the go-around. (Had cbf_rhs been positive, the QP would have found the smallest $\mathbf{u}$ along ∇B to restore safety.)

Steps 9–10 — discrete + analytic safety.

```
v_total = f_theta + (g + u) = (0.101, 0.261, 0)
project_safe : B(x + dt*v_total) = 0.021 >= min(B(x), m) = 0.008  -> keep
analytic_filter : sdf_min(x_next) = 0.015 >= SAFETY_SDF_MARGIN = 0.011 -> ACCEPT
OUTPUT v_safe = (0.101, 0.261, 0.000) m/s   # curve up-and-around the star
```

V.5 The two backstops, formally

$$\textbf{project\_safe:}\ \text{largest } \alpha \in [0,1] \text{ s.t. } B(\mathbf{x} + \alpha\, dt\, \mathbf{v}) \geq \min\big(B(\mathbf{x}), m\big),$$

$$\textbf{analytic\_filter:}\ \text{largest } \alpha \text{ s.t. } \mathrm{sdf}_{\min}(\mathbf{x} + \alpha\, dt\, \mathbf{v}) \geq \texttt{SAFETY\_SDF\_MARGIN} = 0.011.$$

Both use 24 bisection iterations ($2^{-24} \approx 6 \times 10^{-8}$ resolution). The first uses the learned B; the second uses the exact scene SDF as a guaranteed geometric backstop.

Part VI — Sim-to-Real Transfer

VI.1 Why this part matters

Everything above runs in a 2-D kinematic simulation. Deploying on the real FR3 requires that every *dimension, frame, and timing* in simulation matches the physical setup, that the velocity command is realized by a real low-level controller, and that the demonstrations used in sim correspond to what the robot will actually do. This part lists every dimension and gives a complete transfer procedure with the failure modes you will hit and how to fix each one.

VI.2 Global map / workspace dimensions (from config.py)

Quantity	Symbol / key	Value	Meaning
Slab extent X	`SLAB_X`	0.37 – 0.60 m (width 0.23 m)	tissue slab span, robot-base frame
Slab extent Y	`SLAB_Y`	−0.025 – 0.165 m (depth 0.19 m)	lateral span
Needle corridor height	`Z_CORRIDOR`	0.44 m	fixed working plane (z)
Lower foam layer	`Z_BOTTOM`	0.40 m	second foam z-level
Foam sphere radius	`FOAM_RADIUS`	0.018 m	deformable tissue elements
Foam spacing / jitter	`FOAM_SPACING_XY`	0.018 m / ±0.003 m	~188 spheres, 2 z-levels
Start	`X_START`	(0.440, 0.062, 0.440) m	needle entry tip pose
Goal	`X_GOAL`	(0.548, 0.063, 0.440) m	target tip pose
Demo path	`DEMO_WAYPOINTS`	10 pts, arcs to y≈0.091, length ≈0.12 m	expert reference
State dim	`STATE_DIM`	3 (z fixed → planar)	tip position

VI.3 Obstacle dimensions (6 critical zones, 5 shape families)

ID	Family	Label	Center (x,y) m	Key dimensions (m)	Rotation (rad)
C0	star	blocker	(0.500, 0.050)	outer 0.022, inner 0.011, 5 points	+0.3
C1	crescent	vessel	(0.448, 0.120)	outer 0.020, inner 0.013, offset 0.009	—
C2	blob	nerve	(0.470, −0.005)	radii 0.016 / 0.011 / 0.010 (3 lobes)	—
C3	kidney	artery	(0.500, 0.132)	outer 0.019, inner 0.011, squeeze 0.60	+0.8
C4	blob	vein	(0.520, −0.005)	radii 0.014 / 0.010 (2 lobes)	—
C5	lshape	structure	(0.545, 0.120)	arm1 0.028×0.010, arm2 0.010×0.028	−0.3

Bounding radius `CRITICAL_RADIUS` = 0.018 m; safety buffer `CRITICAL_MARGIN` = 0.010 m. Inflation Δ=0.010 m; runtime margins m =0.008, analytic SDF margin 0.011 m.

VI.4 Needle dimensions (physical — must be set to YOUR needle)

The simulation controls only the **tip position** at fixed z; needle geometry is not in config.py because it is a property of your hardware. You must measure and enter the following before deployment. Typical surgical-needle values are given as a starting point.

Quantity	Typical value	How to obtain / why it matters
Needle length L_n	100–200 mm	tip = EE flange pose + L_n along the tool axis; sets the rigid offset
Outer diameter	0.7–1.3 mm (22G–18G)	thinner needles deflect more; affects effective safety radius
Tip offset vector $\mathbf{r}_{\text{tip}}$	measure	transform from flange frame to needle tip (calibrate once)
Bevel / tip type	bevel or symmetric	beveled needles steer asymmetrically in tissue (out of current 2-D model scope)
Stiffness / deflection	measure under load	real tip lags the commanded position; inflate margins to cover it

$$\mathbf{x}_{\text{tip}} = \mathbf{p}_{\text{flange}} + R_{\text{flange}}\,\mathbf{r}_{\text{tip}},$$

where $\mathbf{p}_{\text{flange}}, R_{\text{flange}}$ come from FR3 forward kinematics. This is the quantity that must equal the simulator's $\mathbf{x}$.

VI.5 Control & timing parameters

Quantity	Key	Value	Real-robot note
Control period	`DT`	0.01 s (100 Hz)	match the CRISP / ros2_control loop rate
Velocity clip	`VEL_CLIP`	0.25 m/s	reduce to ~0.02–0.05 m/s for first hardware trials
Episode timeout	`T_MAX`	12 s	watchdog stop
Goal tolerance	`GOAL_TOL`	0.006 m	stop condition
Demo spring	`DEMO_K`	5.0	strong path adherence

VI.6 Step-by-step transfer procedure

```
1. Frame alignment.
   Confirm the simulator base frame == FR3 base frame. Re-express SLAB_X/Y, all obstacle
   centers, X_START, X_GOAL, and the demo path in the real base frame (translate/rotate if
   your tray sits elsewhere). Units: meters.

2. Needle calibration.
   Measure r_tip (flange -> tip). Verify x_tip from FK matches a physical ruler at several poses.

3. Obstacle calibration.
   Photograph / scan the real tissue phantom. For each critical zone, fit one of the 5 shape
   families (center, radii, rotation) OR sample its boundary into a point cloud P_i (K~200).
```

```
   The SAME representation the encoder used must describe the real obstacle.

4. Workspace plane.
   Set z = the real corridor height. Keep the needle in this plane (planar task) unless you
   extend to 3D (Part VI.9).

5. Low-level controller.
   Launch franka_ros2 + CRISP cartesian controller. The policy output v_safe (m/s) is sent as
   a Cartesian velocity / reference-pose increment; CRISP converts it to joint torques.

6. Conservative first run.
   Set VEL_CLIP = 0.02 m/s, raise B_SAFE_MARGIN and SAFETY_SDF_MARGIN by ~50% to absorb
   calibration error and needle deflection. Run with a hand on the e-stop.

7. Closed loop @100 Hz.
   each tick: read joint state -> FK -> x_tip -> compute f, B, gradB, V -> QP -> project_safe
   -> analytic_filter -> send v_safe. Log B, sdf, slack for audit.

8. Tighten gradually.
   Once 0 penetrations are confirmed, lower margins and raise VEL_CLIP toward the sim values.
```

VI.7 The real-time deployment loop (pseudocode)

```
model = load_model("checkpoints/final_model.pt")     # f, V, B + norm stats
ctrl  = BPCBFController()
model.set_obstacles(real_obstacle_clouds)             # calibrated clouds + centers

while not done:                                      # 100 Hz
    q          = read_joint_state()                   # from franka_ros2
    p_flange,R = forward_kinematics(q)
    x_tip      = p_flange + R @ r_tip                 # needle tip in base frame

    s          = progress(x_tip)
    f_val      = model.f(x_tip, s)
    u_safe, info = ctrl.solve(x_tip, model, s=s)      # QP (+ swirl)
    v          = clip(f_val + u_safe, -VEL_CLIP, VEL_CLIP)
    v          = ctrl.project_safe(x_tip, model, v, DT)           # learned-B backstop
    v          = analytic_safety_filter(x_tip, v, DT, real_shapes) # exact-SDF backstop

    send_cartesian_velocity(v)                        # -> CRISP impedance -> torques
    log(info["cbf_val"], info["clf_val"], info["slack"])
    if norm(x_tip - X_GOAL) < GOAL_TOL: done = True
    if info["cbf_val"] < -0.005: emergency_stop()      # inside tissue -> halt
```

VI.8 Calibration checklist (do all before the first run)

- ☐ Base frame of sim == base frame of FR3 (verify a known point with a ruler).
- ☐ Needle tip offset $\mathbf{r}_{\text{tip}}$ measured; FK tip matches physical tip at ≥3 poses.
- ☐ Each real obstacle fitted to a shape family or sampled to a 200-pt cloud, centers in base frame.
- ☐ Working plane z set to the real corridor height; impedance constrains out-of-plane drift.
- ☐ VEL_CLIP reduced to 0.02 m/s; m and SAFETY_SDF_MARGIN raised ~50% for the first trials.
- ☐ 100 Hz loop timing confirmed (measure jitter); e-stop within reach.

- ☐ Logging of B, sdf, slack enabled for the post-run safety audit.

VI.9 What if the real expert demo differs from the trained sim demo?

Short answer: f_θ and the reference path $\mathbf{x}_{\text{ref}}(s)$ are demonstration-specific. If the real task uses a *different* demonstration than the one trained in sim, the demo-flow network will pull the needle toward the *old* path, not the new one.

Three cases:

1. **Same path, small offset/noise.** The $K_f = 5$ spring and CLF absorb it; behavior is fine. The barrier B is unaffected (it is path-independent).
2. **Different path, same obstacles.** Re-run **Stage 1 only** (pretrain f_θ on the new demos, ~30 min Phase 0; or full Stage 1 if you also want V retuned). The barrier B can be **reused as-is** — it depends only on obstacle geometry, not on the path.
3. **Different obstacles too.** Recalibrate obstacle clouds/SDF (VI.6 step 3). Thanks to pose/scale augmentation, the existing B generalizes zero-shot to new poses/scales of the *same 5 families*. A genuinely new shape family needs a Stage 2 fast retrain (~5 min) with that family added.

Practical rule: **swap demos → retrain f (and maybe V); swap obstacle poses → nothing; swap obstacle *shapes* → fast-retrain B.** This modularity is a direct benefit of separating f, V, B.

VI.10 Sim-to-real gaps: issues and fixes

#	Gap / Issue	Symptom	Fix
1	Frame mismatch	needle aims at wrong region	re-express all geometry in FR3 base frame; verify with FK at known poses
2	Needle deflection / tissue reaction	real tip lags command, grazes obstacle	raise m and SAFETY_SDF_MARGIN; lower VEL_CLIP; add deflection model to $\mathbf{r}_{\text{tip}}$
3	Foam deforms (sim foam is rigid spheres)	obstacle effectively shifts under load	treat as slow-moving obstacle; re-evaluate B each step (already supported); add margin
4	Obstacle pose error in calibration	near-miss on one side	10 mm inflation covers ≤10 mm error; improve vision; inflate further if uncertain
5	Velocity command realization	CRISP cannot track high $\dot{\mathbf{x}}$	cap VEL_CLIP to tracked range; tune impedance gains; check 100 Hz timing
6	Latency / jitter in the loop	discrete-time overshoot into obstacle	project_safe + analytic_filter handle finite steps; keep loop jitter low; reduce dt if possible
7	Tip-position sensing	FK-only tip ignores in-tissue bending	external tracking (EM/optical) or a bending model; fuse with FK

8	2-D model vs 3-D reality	out-of-plane drift	constrain z by impedance; or extend to 3-D clouds/SDF (VI.9)
9	Demo mismatch	needle pulled off the intended path	retrain f_θ on the real demos (VI.7)
10	Barrier "compression"	standoff larger than nominal (25–40 mm)	account for it in margin budget; or improve B edge accuracy

VI.11 Extending to true 3-D (future)

The cleanest extension: make clouds $P_i \in \mathbb{R}^{K\times 3}$, the encoder $\mathrm{Linear}(3 \to 128)$, the CBF input $3 + 64 = 67 \to 4 + 64 = 68$? No — $\mathbf{x}_{\text{rel}}$ becomes 3-D so the input is $3 + 64$ already in code (STATE_DIM=3); only the cloud's per-point input changes from 2 to 3. Provide 3-D SDF labels and 3-D demos. Everything else (QP, smooth-min, augmentation with $SO(3)$ rotations) carries over.

Part VII — Literature map (what each /docs paper contributed)

Paper	Contribution used here
Learning Complex Motion Plans using Neural ODEs (Nawaz et al. 2023)	Core template: progress-conditioned f_θ, quadratic CLF base $V = \|e\|^2$, CLF–CBF QP (min $\|u\|^2 + \lambda\varepsilon^2$), parameters $\alpha = 4, \gamma = 3, \lambda = 0.5$, RK4 imitation loss, pretrain-then-jointly-train flow.
CN-CBF (Composite Neural CBF)	Smooth-min composition $B = -\frac{1}{\beta}\log\sum e^{-\beta b_i}$ (Eq. 18), PointNet-style permutation-invariant encoder, shared per-obstacle conditional CBF, β vs error bound $\log N/\beta$, dynamic re-evaluation each step.
Safe & Stable NN Dynamical Systems (S2-NNDS)	Algorithm 2 joint training of f, V, B with counter-example refinement; learned barrier (no analytic anchor); sign convention $B > \delta$ safe, $B < -\delta$ unsafe; discrete projection idea.
Learning CBFs from Expert Demonstrations	SDF labels as supervised barrier targets; hinge loss for unsafe points; counter-example refinement.
Sequential Neural Barriers for Scalable Dynamic Obstacle Avoidance	Composing many barrier certificates; dynamic re-evaluation; weight sharing across obstacles.
Survey on CLF & CBF for Nonlinear-Affine Systems	Formal CBF/CLF definitions, class-$\mathcal{K}$ (exponential) CBF condition $\nabla B^\top(f+u) \geq -\gamma B$, QP feasibility and the role of γ.
Learning Robust Output CBFs from Safe Expert	Robust margins / buffers (our m); robustness to model error.
Beik-Mohammadi: Extended Neural Contractive Dynamical Systems	Progress-conditioned velocity fields, demo interpolation $\mathbf{x}_{\text{ref}}(s)$, feedback correction $K(\mathbf{x}_{\text{ref}} - \mathbf{x})$.
BP_CBF_Technical_Pitch	Original problem framing, the (later-removed) penetration-budget idea, the hybrid learned+analytic safety-filter concept.

Consulted but not directly implemented: HJ-Reachability set, RAIL, V-OCBF, Collision-Cone CBF, Online Adaptive High-Order Safety — informed alternatives (reachability backstops, online updates, velocity-obstacle CBFs) we chose not to adopt for real-time simplicity.

Part VIII — Honest ICRA assessment

VIII.1 Genuine strengths

1. Pose-generalizing composite learned CBF: PointNet encoding + smooth-min + pose augmentation for zero-shot generalization is a clean, novel combination.
2. Two-stage training (freeze f, V; fast-retrain B in ~5 min) is practical and deployable.
3. Hybrid safety filter: learned field for behavior, exact SDF for the guarantee.
4. Works on hard non-convex shapes (star tips, crescent horns, kidney concavity) — most CBF papers use circles or convex polytopes.
5. SDF-shaped target with asymmetric clamp is a subtle, well-motivated design choice.

VIII.2 Weaknesses likely to draw rejection

1. **No hardware experiments** — the single biggest gap for a robotics venue.
2. **2-D task in a 3-D world** (fixed z); reviewers will ask about true 3-D needle insertion.
3. **No baselines** (fixed-pose CBF, analytic-SDF CBF, a clutter method like "Learning to Nudge").
4. **Hybrid filter can look ad-hoc**: "if you have the exact SDF, why learn?" needs a crisp answer (SDF unknown at deployment; learned field gives better trajectories).
5. **Barrier compression** (slope ≈1.5 vs target $K = 4$) → larger-than-claimed standoff; not fully explained.
6. **Few formal guarantees** beyond the analytic backstop; the learned part is empirical.
7. **Small evaluation** (5 families, ~8 configs); ICRA expects statistics with error bars.

VIII.3 Improvement roadmap

Must do: (1) FR3 hardware experiments with measured penetration; (2) baselines (fixed-pose CBF, analytic-SDF CBF, clutter method); (3) crisp contribution statement.
Should do: (4) fix or formally account for barrier compression; (5) 50+ randomized configs with confidence intervals; (6) a robustness bound relating gradient error to safe margin.
Nice to have: (7) full 3-D; (8) online Stage-2 retraining for moving obstacles; (9) comparison on a standard benchmark.

Verdict: not yet ICRA-ready; ~3–6 months of work (hardware + baselines + writing). Interim: an ICRA workshop / RA-L submission is realistic.

VIII.4 A concrete experiment design

To turn this into a defensible submission, run this matrix and report every cell with mean ± std over ≥50 randomized scenes.

Method	Penetration rate	Success (reach)	Path deviation	Solve time
Ours (learned composite CBF + hybrid filter)	target 0%	report	report	<1 ms
Fixed-pose CBF (no augmentation)	expected high at rotated poses	report	report	<1 ms
Analytic-SDF CBF (no learning; needs known SDF)	0% when SDF known	report	report	<1 ms
Potential-field / "Nudge" clutter baseline	report	report	report	report
No safety filter (demo flow only)	high	high	0	—

Axes to sweep: obstacle count (1–12), rotation (0–2π), scale (0.6–1.5), on-path vs off-path, static vs dynamic. Ablations: remove augmentation; remove eikonal; remove analytic filter; remove swirl. Each ablation should visibly degrade one metric — that is what convinces reviewers each component earns its place.

Part IX — Q&A

Conceptual

Q. One-sentence summary?
A. Steer a needle from start to goal through foam, strictly avoiding non-convex tissue regions that may appear at unseen poses/scales, in real time, using a learned demo flow + learned barrier + a QP.

Q. What is a CBF, intuitively?
A. A scalar "force field" $B(\mathbf{x})$: far from obstacles it relaxes; near them it stiffens and forces velocity to point away. If you start safe ($B \geq 0$) and respect $\dot{B} \geq -\gamma B$, you stay safe forever.

Q. When does the QP "activate"?
A. Every 10 ms, but it returns $\mathbf{u} \approx 0$ when the demo flow is already safe. It only corrects when f_θ would violate the CBF.

Technical

Q. The CBF MLP input is 67-D including the embedding — does one network handle all shapes?
A. Yes. One shared MLP; the obstacle's identity enters via the 64-D embedding and the relative position. It learns "given relative position and shape $\mathbf{e}_i$, how far inside/outside am I?"

Q. Why 200 cloud points?
A. Coverage vs speed. 200×(up to 6 obstacles) per step is affordable at 100 Hz; 20 misses thin tips, 1000 is wasteful.

Q. Why no activation after the last per-point layer?
A. So max-pool selects the true maximum pre-activation feature; non-linearity resumes in the CBF MLP's first tanh layer.

Q. Why use log-sum-exp instead of `min`?
A. min is non-differentiable where two b_i tie; the QP needs ∇B. Log-sum-exp is smooth and $\to \min$ as $\beta \to \infty$; it is computed stably as $\text{logsumexp}(x) = \max(x) + \log \sum e^{x - \max(x)}$ to avoid overflow.

Mathematical

Q. Show $B \leq \min_i b_i$.
A. Since $e^{-\beta m} \leq \sum_i e^{-\beta b_i}$ with $m = \min_i b_i$, taking log and dividing by $-\beta$ gives $B = -\frac{1}{\beta} \log \sum \leq m$. Combined

with the lower bound, $m - \frac{\log N}{\beta} \leq B \leq m$. For N=6, β=1000: within ≈1.8 mm.

Q. Why does $\dot{V} \leq -\alpha V$ imply convergence?
A. $\frac{d}{dt} \log V \leq -\alpha \Rightarrow V(t) \leq V(0) e^{-\alpha t}$ (Grönwall). With $\alpha = 4$, error halves every $\log 2/4 \approx 0.17$ s.

Q. What class of function is γB?
A. A class-$\mathcal{K}$ (linear) function of B — an "exponential CBF." On the boundary ($B = 0$) it forces $\dot{B} \geq 0$; in the interior it permits decay no faster than $e^{-\gamma t}$.

Application

Q. A brand-new obstacle *shape* appears — what happens?
A. The encoder embedding goes out-of-distribution, so the learned field may navigate poorly, but the analytic SDF backstop still prevents penetration. Fix: add that shape and run the 5-min Stage-2 retrain.

Q. Fast-moving obstacle (0.5 m/s)?
A. Predictive look-ahead (0.10 s) helps for slow motion; very fast obstacles may close the margin faster than γ allows — needs higher γ or a higher-order CBF (future work).

Q. Why is the barrier "SDF-shaped" rather than a 0/1 classifier?
A. The QP uses ∇B. A step function has $\nabla B \approx 0$ away from the edge, so the constraint becomes trivial or infeasible. An SDF-shaped B has $\|\nabla B\| \approx K = 4$ everywhere, giving a usable push direction at any distance.

Q. Explain the head-on deadlock fix.
A. Head-on, ∇B directly opposes f_θ; the QP cancels to ≈ 0 and the needle stalls. Adding a tangential swirl $\mathbf{g} \perp \nabla B$ (toward the goal) lets the needle circle the obstacle without violating the CBF; the CLF slack absorbs the temporary error.

Q. What exactly is m, and is it the same as the eikonal term?
A. No. $m =$ `B_SAFE_MARGIN` =0.008 is a *runtime* margin: the QP defends $\{B \geq m\}$ instead of $\{B \geq 0\}$ so residual barrier error cannot push the needle past the true boundary. The eikonal term is a *training* regularizer that caps $\|\nabla B\|$. One acts at inference, the other during learning.

Q. Why is the CLF "soft" but the CBF "hard"?
A. Violating the CBF means hitting tissue — unacceptable, so it is a hard inequality. Violating the CLF only means converging to the path more slowly — acceptable, so it is relaxed by slack ε (penalized by $\lambda\varepsilon^2$) to keep the QP feasible when avoiding an obstacle conflicts with following the path.

Q. How many parameters does the whole system have?

A. Roughly $f \approx 34\text{k}$, $V \approx 4.5\text{k}$, encoder $\approx 25\text{k}$, conditional CBF $\approx 149\text{k} \rightarrow$ on the order of 2.1×10^5 trainable parameters. Small enough to run the forward passes and the QP at 100 Hz.

Q. Does project_safe ever make the robot stop?

A. It scales the step by $\alpha \in [0, 1]$. If no forward fraction keeps $B \geq \min(B(x), m)$, it returns $\alpha = 0$ (stop) — but because it preserves direction, tangential / outward motion still survives, so the needle slides around rather than freezing, except in a genuine pocket trap.

Q. Why 6 obstacles but "5 shapes"?

A. There are five shape families (star, crescent, blob, kidney, L-shape); the blob family is instantiated twice (nerve C2 and vein C4), giving six physical obstacles in the canonical scene.

Q. One-minute pitch to a non-expert professor.

A. "We train three small neural networks — one to follow demonstrations, one to certify convergence, one to certify safety against irregular obstacles — using random rotations and rescalings so the safety net understands shapes rather than memorizing poses. At run time we solve a tiny optimization 100×/s to pick the safest velocity that still follows the demonstration, with an exact geometric backstop. It achieves zero penetrations across all tested cases, including never-seen obstacle poses."

Appendix A — The exact obstacle SDF formulas

These are the analytic signed-distance functions used to generate barrier targets and to drive the exact safety backstop. Each takes a query point $\mathbf{p}$, subtracts the center $\mathbf{c}$, and (for rotated shapes) applies the inverse rotation $R(-\rho) = \left(\begin{smallmatrix} \cos\rho & \sin\rho \\ -\sin\rho & \cos\rho \end{smallmatrix}\right)$ to get local coordinates (p_x, p_y).

A.1 Star (C0 "blocker")

$$r = \sqrt{p_x^2 + p_y^2}, \quad \theta = \operatorname{atan2}(p_y, p_x), \quad \text{sector} = \tfrac{\pi}{n}, \quad a = \big(\theta \bmod 2\,\text{sector}\big) - \text{sector},$$

$$r_{\text{star}} = r_{\text{in}} + (r_{\text{out}} - r_{\text{in}})\Big(1 - \tfrac{|a|}{\text{sector}}\Big), \qquad \text{sdf} = r - r_{\text{star}}.$$

$n = 5$, $r_{\text{out}} = 0.022$, $r_{\text{in}} = 0.011$, $\rho = 0.3$. The angular term carves the five concave notches.

A.2 Crescent (C1 "vessel")

$$\text{sdf} = \max\big(\underbrace{r_{\text{out}}^{\,p} - R_{\text{out}}}_{\text{inside big disk}}, \ \underbrace{R_{\text{in}} - r_{\text{in}}^{\,p}}_{\text{outside the bite}}\big),$$

$r_{\text{out}}^{\,p} = \|\mathbf{p} - \mathbf{c}\|$, $r_{\text{in}}^{\,p} = \|\mathbf{p} - (\mathbf{c} + \text{offset})\|$. A big disk minus an offset disk = a crescent with two thin horns.

A.3 Blob (C2 "nerve", C4 "vein") — union of disks

$$\text{sdf} = \min_k \Big(\|\mathbf{p} - (\mathbf{c} + \text{off}_k)\| - r_k\Big).$$

C2: radii $(0.016, 0.011, 0.010)$, 3 lobes. C4: radii $(0.014, 0.010)$, 2 lobes.

A.4 Kidney (C3 "artery")

$$\text{sdf} = \max\Big(\sqrt{p_x^2 + (p_y/q)^2} - r_{\text{out}}, \ \ r_{\text{in}} - \sqrt{(p_x - 0.4\,r_{\text{out}})^2 + p_y^2}\Big),$$

squeeze $q = 0.60$, $r_{\text{out}} = 0.019$, $r_{\text{in}} = 0.011$, $\rho = 0.8$. An ellipse with an off-center disk subtracted = the kidney's concavity.

A.5 L-shape (C5 "structure") — union of two rectangles

$$\text{rect}(q_x, q_y, h_w, h_h) = \big\|\big(\max(|q_x| - h_w, 0), \max(|q_y| - h_h, 0)\big)\big\| + \min\big(\max(|q_x| - h_w, |q_y| - h_h), 0\big),$$

$$\text{sdf} = \min\big(\text{rect}_1, \ \text{rect}_2\big).$$

arm1 $= 0.028 \times 0.010$, arm2 $= 0.010 \times 0.028$, $\rho = -0.3$. The standard box SDF, combined as an "L".

Appendix B — Complete parameter reference (config.py)

Group	Name	Value	Role
Geometry	SLAB_X / SLAB_Y	(0.37,0.60) / (−0.025,0.165)	workspace extents (m)
	Z_CORRIDOR / Z_BOTTOM	0.44 / 0.40	working plane / lower foam (m)
	X_START / X_GOAL	(0.440,0.062,0.44)/(0.548,0.063,0.44)	endpoints
	FOAM_RADIUS	0.018	foam sphere radius (m)
	CRITICAL_RADIUS	0.018	obstacle bounding radius (m)
	CRITICAL_MARGIN	0.010	buffer beyond boundary (m)
Networks	F_HIDDEN	[128,128,128]	demo-flow MLP widths
	V_HIDDEN	[64,64]	Lyapunov correction widths
	EMB_DIM	64	PointNet embedding size
	N_POINTS	200	cloud points per obstacle
Barrier	INFLATE_MARGIN (Δ)	0.010	B=0 sits 10 mm outside tissue
	BARRIER_SDF_K (K)	4.0	SDF target slope
	BARRIER_SDF_CLAMP_OUT/IN	0.013 / 0.040	asymmetric target clamp
	BARRIER_BETA (β)	1000	smooth-min sharpness
	B_GRAD_MAX	12.0	eikonal gradient cap
	LAMBDA_EIK	0.02	eikonal weight
Control	GAMMA_CBF (γ)	3.0	CBF decay rate
	ALPHA_CLF (α)	4.0	CLF convergence rate
	LAMBDA_SLACK (λ)	0.5	CLF slack penalty
	B_SAFE_MARGIN (m)	0.008	defended barrier level
	SAFETY_SDF_MARGIN	0.011	analytic backstop distance
	DEMO_K / SWIRL_GAIN / B_ACTIVE	5.0 / 1.0 / 0.040	spring / swirl / swirl trigger
Timing	DT	0.01	100 Hz period (s)
	VEL_CLIP	0.25	max speed (m/s)
	T_MAX / GOAL_TOL	12 / 0.006	timeout (s) / goal radius (m)

	OUTER_ITERS / INNER_EPOCHS	10 / 150	Stage-1 loop sizes
Training wts	LAMBDA_MSE	50	imitation rollout
	LAMBDA_B_FS / LAMBDA_B_OB	5 / 12	safe regression / unsafe hinge
	LAMBDA_B_REG / N_CEX	20 / 2000	SDF regression / counter-examples

Appendix C — File & pipeline map

File	Purpose
src/config.py	all geometry, parameters, shape SDFs, augmentation helpers
src/models.py	ProgressConditionedDS (f), LyapunovNet (V), CompositeBarrier (B)
src/train.py	Stage-1 joint training (S2-NNDS Algorithm 2 + CEX)
src/train_barrier_fast.py	Stage-2 barrier-only augmented training
src/cbf_qp.py	online QP, project_safe, analytic_safety_filter
src/simulate.py	single rollouts + field/barrier plots
src/run_rollouts.py	normal / generalized / dynamical rollout batches
src/generalization_test.py	zero-shot pose/scale scenes (validated)
src/dynamic_env_test.py	moving/rotating/growing obstacle tests
src/make_final.py	paper-quality figures + GIFs
run.md	all commands from venv to figures

— End of Document —

Source: /home/stanny/franka_ros2_ws/src/Tolerance_Aware_CBF(TA-CBF)/